Name - Zeeya
Roll No - 23293916004
Cloud Computing
Assignment - 2

Date .....

1. Explain the architecture of a Helm chart, including its main components.

Ans→ A Helm Chart is a packaged Kubernetes application. Key components:

- Chart.yaml - metadata: name, version, description
- values.yaml - default config parameters (overridable at deploy time)
- templates/ - Go - templated Kubernates YAML files (Deployments, Services etc.)
- Charts/ - ~~Custom Resource~~ sub - chart dependencies (e.g. Redis, Postgre SQL)
- crds/ - Custom Resource Definitions to install before templates
- NOTES.txt - post - install instruction printed to terminal.

Flow: values.yaml + templates/ → Helm engine renders → pure Kubernetes YAML → applied to cluster.

2. Discuss the advantages and limitations of serverless (FaaS).

Ans→ Advantages:

- No server management: provider handles infra
- Auto - scales from zero to thousands of instances
- Pay - per - execution (no idle cost)
- Fast deployment and event - driven integration

Spiral

Date .....

Limitations:

- Cold start latency (100 ms – 2s on first invocation)
- Execution time limits (AWS Lambda max: 15 min)
- Stateless by design; state needs external storage
- Vendor lock-in; hard to migrate
- Difficult to debug / trace across many short-lived invocations.

3. Explain how Google BigQuery handles large-scale analytics efficiently.

Ans→ BigQuery achieves efficiency through:

- Columnar storage (Capacitor) – reads only queried columns, minimizing I/O
- Dremel architecture – query tree fans out to thousands of parallel leaf nodes; aggregations happen at each level as results travel upward
- Separation of storage and compute – scales independently; thousands of slots execute in parallel.
- Query optimizer – predicate pushdown, join reordering, broadcast vs shuffle join shuffle
- Partitioning & clustering – partition pruning skips entire data / range partitions; clustering sorts data for faster scans
- BI Engine – in memory caching for sub-second repeated queries

Date .....

4. Compare :
- Pre-built ML APIs
- AutoML
- Custom ML models

Ans→

| | Pre-built APIs | AutoML | Custom Models |
|---|---|---|---|
| Skill needed | None | Low | High |
| Data needed | None | Hundred-thousand | Millions |
| Customization | None | Moderate | Full |
| Time to deploy | Minutes | Hours-days | Weeks-mont |
| Best for | Generic tasks | Domain-specific | Novel/ resear |
| Example | Vision API, Translate | AutoML Tables | Tensorflow Pytorch |

5. Explain the complete lifecycle of a serverless function, including trigger, execution and scaling.

Ans→ 1. Deploy - upload code, set runtime, memory, timeout
2. Register trigger - HTTP, queue event, storag event, cron
3. Trigger fires - invocation event received by platform.
4. Cold / warm start - new container initializ (cold) or existing reused (warm)
5. Execute - handler runs with event payload, accesses external services
6. Return response - result sent back to caller

Spiral

Date .....

7. Auto-scale - concurrent events spawn parallel instances automatically
8. Idle / teardown - container frozen or GC'd after inactivity; billing stops

6. Discuss the role of Helm in DevOps and CI/CD pipelines.

Ans → Core roles:
- Templating - single chart deploys to dev/staging/prod via different values.yaml
- Release tracking - every deploy is a versioned revision; helm rollback reverts instantly
- Dependency management - declares all app dependencies in chart.yaml

CI/CD pipeline flow:
1. code push → Docker image built and pushed to registry
2. helm package → chart pushed to Helm repo
3. helm upgrade -- install with enviroment specific values file
4. helm test runs in-cluster smoke tests
5. On failure: helm rollback restores previous revision automatically

GitOps: Tools like ArgoCD / Flux watch Git for chart changes and auto-sync the cluster, making Helm charts the deployment artifact and Git the source of truth.

Spiral

Date .....

7. Explain the working of edge inference with a step-by-step process.

Ans→1. Train - full model trained on cloud GPU
2. Optimize - quantization (FP32 → INT8); pruning, knowledge distillation
3. Convert - export to edge format : TF Lite, ONNX, Core ML, Tensor RT
4. Deploy - model pushed to device via OTA update flashing
5. Capture - sensor collects raw data (camera frame audio, IMU)
6. Preprocess - resize, normalize, extract features on-device CPU / DSP
7. Infer - model forward pass runs on NPU / GPU / DSP locally
8. Postprocess - raw outputs → labels, bounding boxes, anomaly scores → action
9. Telemetry - only metadata / results sent to cloud; raw data never leaves device

Key benefit : No network round trip → sub-millisecond latency, offline capability, data privacy.